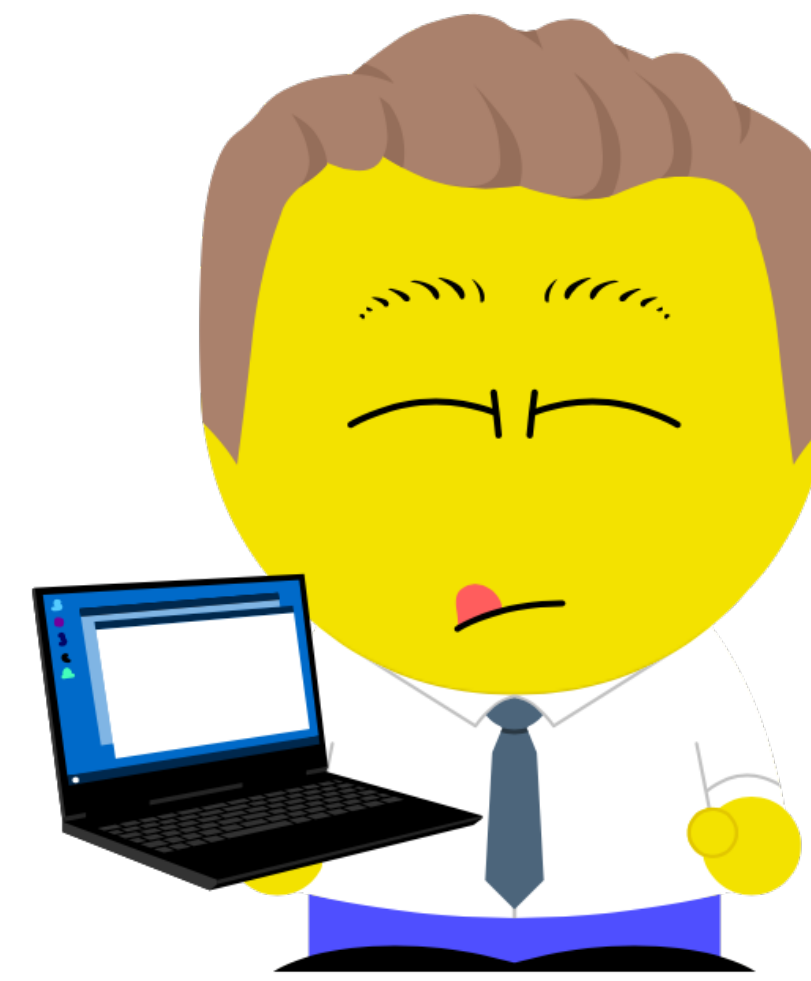




IN-CLASS EXERCISES



OO LANGUAGES 1

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`oo-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip oo-1-exercises.zip
```

```
> cd oo-1-exercises
```

```
> ls
```

Task 1

- Explain these terms:

1. HAS-A vs. IS-A

2. Late Binding

3. Inclusion Polymorphism

4. Runtime Type Checks

5. Dynamic Dispatch

Task 2

- Draw a figure that describes the state of this program at point <<HERE>>. It should have one element for each item stored in memory (i.e. global/heap variables, templates, method object code, etc.) and should explicitly describe what each pointer points to.

```
class T {  
    int v;  
    char c;  
    void P(int x) {}  
    void Q(char x) {}  
}
```

```
class U extends T {  
    float x;  
    int k;  
    void R(int x) {}  
    void Q(float x) {}  
}
```

```
void main(...) {  
    T t;  
    U u;  
    t = new T();  
    u = new U();  
    <<HERE>>  
}
```

Task 3

- What happens when both a class and its subclass have an instance variable with the same name?
- Use this code to help you answer!

```
class C1 {int a;}
class C2 extends C1 {double a;}
class C {
    static public void main(String[] arg) {
        C1 x = new C1(); C2 y = new C2();
        x.a = 5; y.a = 5.5;
        ((C1)y).a = 5;
    }
}
```

Task 4

- Translate main below to C.
- Make sure to include the appropriate templates pointers, method lookups, etc.

```
void main (String args[]) {  
    Square q;  
    q = new Square();  
    q.x = 1; q.y = 3; q.side = 15;  
    q.draw();  
    q.move(20, 30);  
}
```

```
class Shape {  
    int x, y;  
    void draw() { }  
    void move(int X, int Y) { }  
}  
class Square extends Shape {  
    int side;  
    void draw() { }  
}  
class Circle extends Shape {  
    int radius;  
    void draw() { }  
    int area() { return 1; }  
}
```


Task 5

- What are the results (*OK compile-time check, failed compile-time check, OK runtime check, failed runtime check*) of the assignments below?

```
Shape s = new Square();
Shape t = new Circle();
Square q = new Square();
Circle c = new Circle();
s = c;
s = q;
c = q;
c = (Circle)s;
q = c;
q = (Square)s;
q = (Square)t;
```

```
class Shape {
    int x, y;
    void draw() { }
    void move(int X, int Y) { }
}
class Square extends Shape {
    int side;
    void draw() { }
}
class Circle extends Shape {
    int radius;
    void draw() { }
    int area() { return 1; }
}
```

Task 6

1. What is the output of the program below?
2. Draw the class hierarchy.
3. Draw the template data structure.
4. What C code will "c instanceof T" be translated into?

```
void main(String[] args) {  
    C c = new C();  
    T s = new S();  
    if (c instanceof T)  
        println("C");  
    if (c instanceof S)  
        println("S");  
    if (s instanceof U)  
        println("U");  
}
```

```
class T {}  
class S extends T {}  
class A extends S {}  
class B extends S {}  
class C extends B {}  
class U extends T {}  
class V extends U {}  
class Y extends U {}  
class Z extends Y {}
```


Task 7

1. Draw the layout of objects of type T and U. Include all instance variable, template pointers, etc., and show their offsets. In Java, an `int` is 4 bytes and a `double` is 8 bytes.
2. Show the C code that method `U.P` will be translated into.
3. Show the C code that the method invocations in `main` (`u.P()` and `u.Q()`) will be translated into.

```
class T {  
    int x;  
    void P(int x) {  
        this.x = x;  
    }  
}
```

```
class U extends T {  
    double x;  
    void P(int x) {  
        super.x = x;  
    }  
    void Q(double x) {  
        this.x = x;  
    }  
}
```

```
void main(...) {  
    U u = new U();  
    u.P(10);  
    u.Q(10.0);  
}
```

Task 8

- Say that you want to inline the call to method `P()` in `main` below. You don't know what type the `t` object will have at runtime.
- How would you do it?

```
void main(String[] args) {  
    T t;  
    if (args[1].equals("S"))  
        t = new S();  
    else  
        t = new U();  
    t.P();  
}
```

```
class T {  
    void P() {  
        System.out.println("T");  
    }  
}  
class S extends T {  
    void P() {  
        System.out.println("S");  
    }  
}  
class U extends T {  
    void P() {  
        System.out.println("U");  
    }  
}
```

Task 9 (a)

- The iterative `ISTYPE` runtime type checking function follows an object's template pointer and the templates' parent pointers up towards the root of the tree.

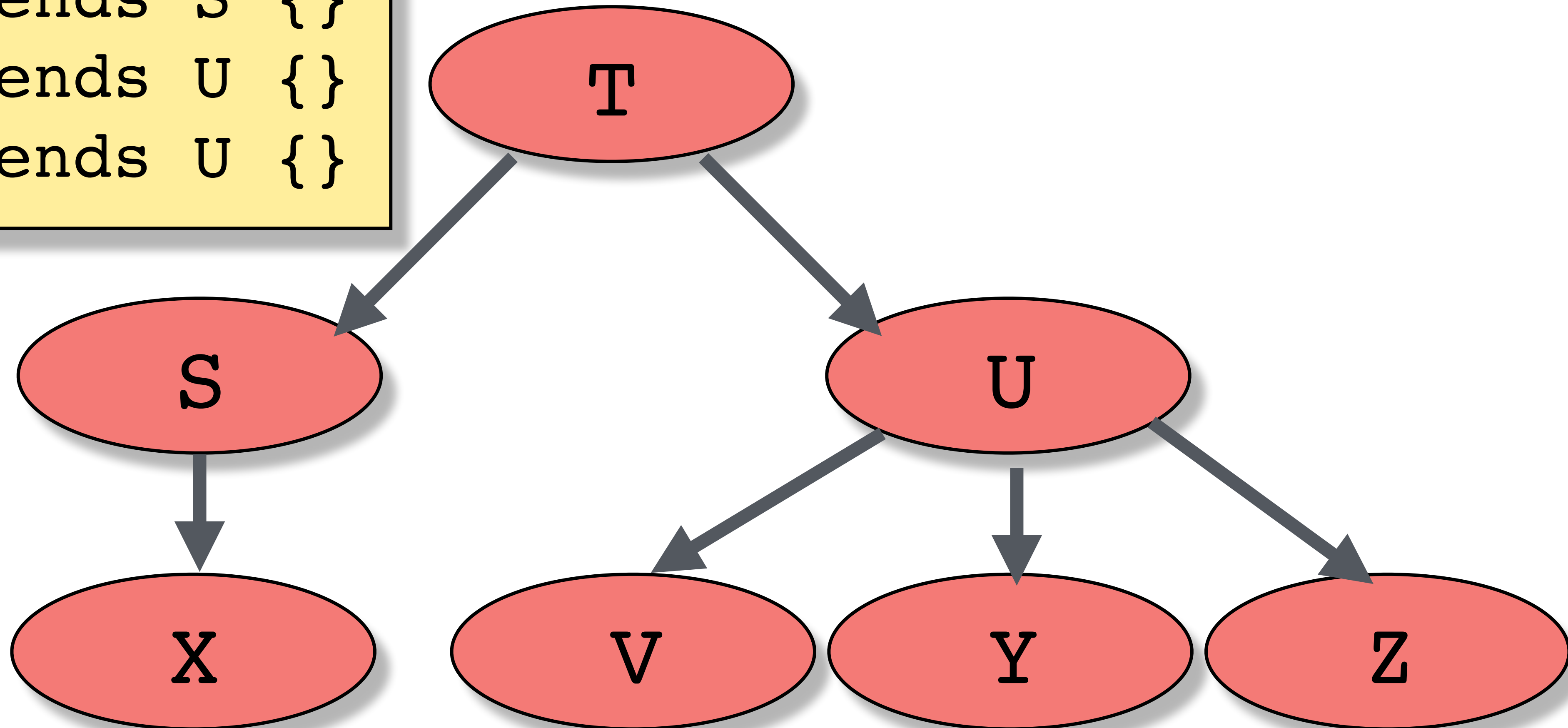
```
bool ISTYPE_n (Template s, t) {  
    while(1) {  
        if (s == t) return true;  
        s = s->parent;  
        if (s == root) return false;  
    }  
}
```

- `ISTYPE` is $O(n)$ in the worst case, where n is the height of the inheritance hierarchy.

Task 9 (b)

```
class T {}  
class S extends T {}  
class U extends T {}  
class V extends U {}  
class X extends S {}  
class Y extends U {}  
class Z extends U {}
```

Inheritance
Hierarchy



•Here's an
example:

Task 9 (c)

- There is an alternative algorithm where type checks run in constant time:
 1. Build the inheritance tree.
 2. Perform a preorder traversal and assign preorder numbers to each node.
 3. Similarly, assign postorder numbers to each node.
 4. Store T's pre- and postorder numbers in T's template.

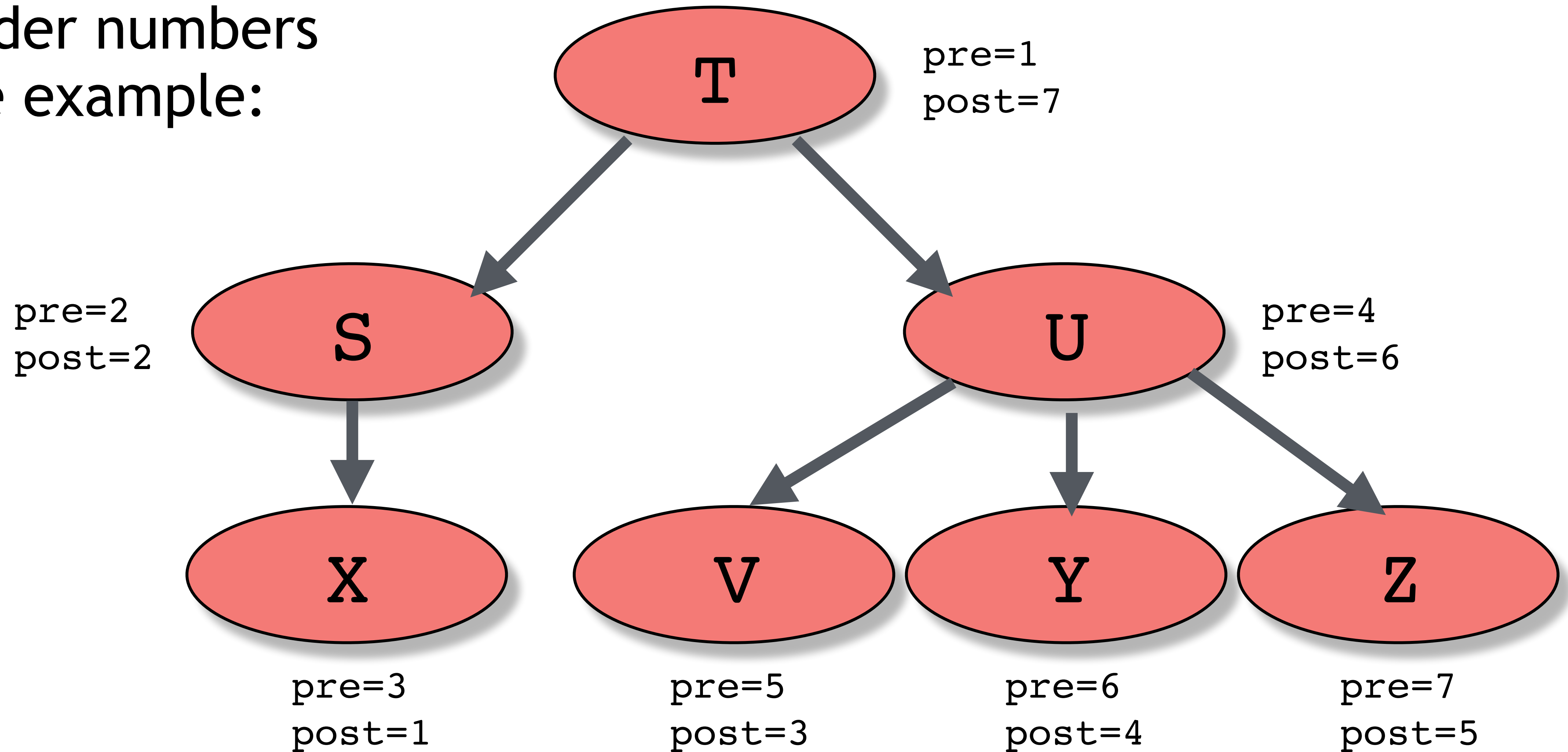
Task 9 (d)

- Given the pre and postorder numbers, we can now compute `ISTYPE` in constant time:

```
bool ISTYPE_1 (Template s, t) {  
    return ((s->pre <= s->pre) &&  
            (t->post >= s->post));  
}
```


Task 9 (e)

- Here are the pre and postorder numbers for the example:



Task 9 (f)

- Using both algorithms `ISTYPE_n` and `ISTYPE_1`, compute
 1. `ISTYPE ( Y , U )`
 2. `ISTYPE ( Z , S )`
 3. `ISTYPE ( Z , T )`
- Do you get the same result for both algorithms?
- On a scale from 1 to 10, 10 being highest, how cool and surprising is this algorithm?



DEFINITIONS AND ALGORITHMS

Method Lookup Algorithm

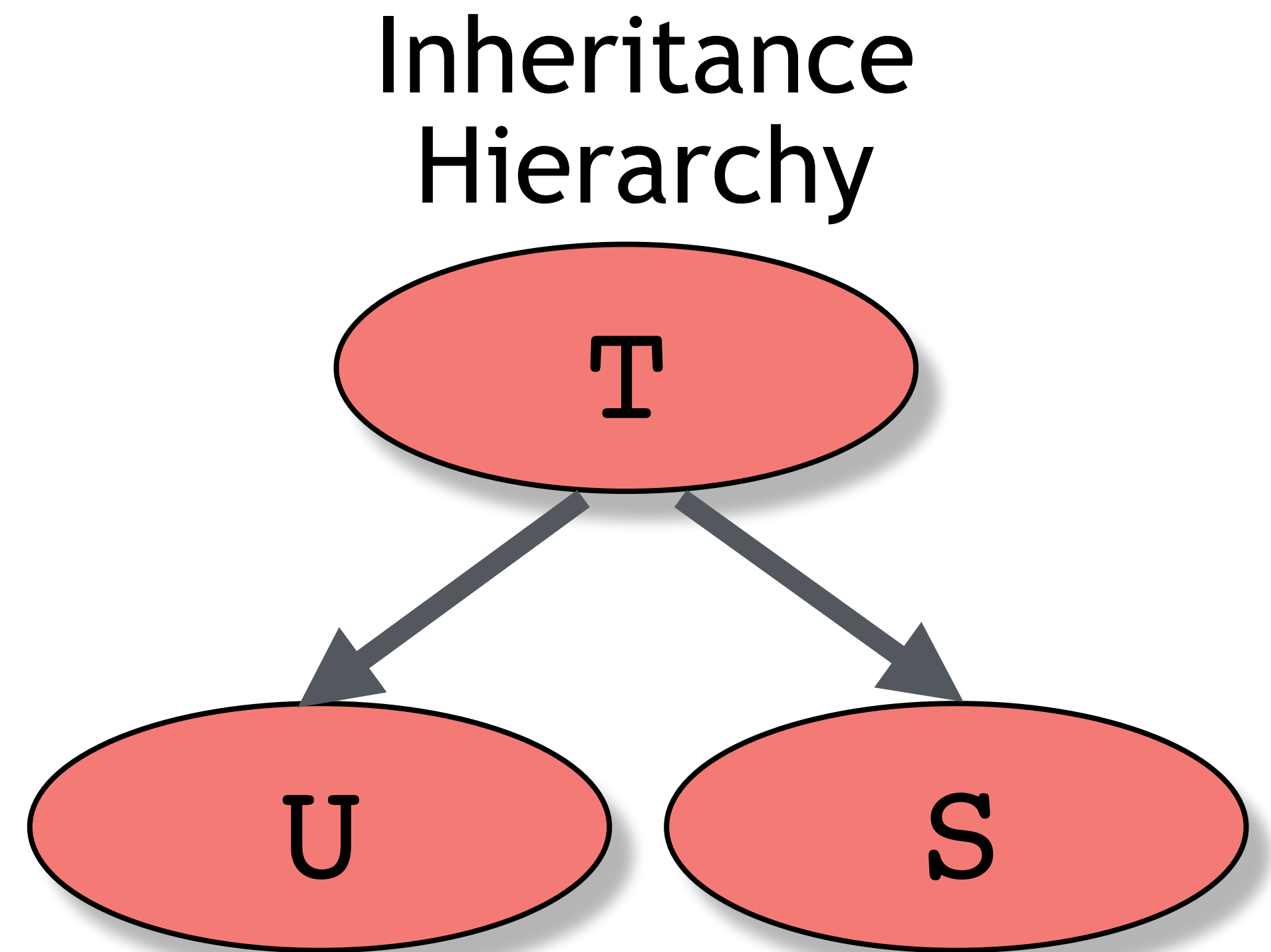
- Sending the message `draw` to `q` (e.g. call `q.move()`):
 1. Follow the link to `q`'s template, `T`.
 2. Get `draw`'s address from `T`.
 3. Jump to `draw`'s address, with `q` as the first (`this`) argument.

```
bool ISTYPE (Template s, t) {  
    while(1) {  
        if (s == t) return true;  
        s = s->parent;  
        if (s == root) return false;  
    }  
}
```



```
class T {}  
class U extends T {}  
class S extends T {}
```

```
T t;  
U u;  
S s;
```



Assignment	Compile-time	Runtime
t = s	Legal	Legal
t = u	Legal	Legal
u = t	Legal	Check
s = u	Illegal	

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA